

Lab Session: Doubly Linked List

September 26, 2024

Objective

In this lab session, we will understand the process of creating a doubly linked list, along with performing common operations on it such as insertion and deletion. By the end of this lab, you should be able to:

- Understand the structure of a doubly linked list.
- Be able to traverse the list in both directions (forward and backward).
- Implement basic operations like inserting and deleting nodes.

What is a Doubly Linked List?

A doubly linked list is a type of linked list where each node contains three fields:

- **Data:** The value stored in the node.
- **Next Pointer:** A pointer to the next node in the list.
- **Previous Pointer:** A pointer to the previous node in the list.

This enables traversal in both forward and backward directions, unlike singly linked lists where traversal is only possible in one direction.

Steps to Create a Doubly Linked List

Follow the steps below to create a doubly linked list:

Step 1: Define the Structure of a Node

Each node in a doubly linked list will have the following components:

- A data field to store the value.
- A pointer to the next node.
- A pointer to the previous node.

Step 2: Initialize the List

Begin by initializing the head and tail of the doubly linked list to NULL. This represents an empty list.

Step 3: Insert a Node at the Beginning

Design an algorithm to insert a node at the beginning of the list. Write these steps in the notebook and provide the implementation.

Step 4: Insert a Node at the End

Design an algorithm to insert a node at the end of the list. Write these steps in the notebook and provide the implementation.

Step 5: Insert a Node at a Specific Position

Design an algorithm to insert a node at a specific position in the doubly linked list. Write these steps in the notebook and provide the implementation. **Note:** Be careful with edge cases, such as inserting at the head (position 1) or at the tail of the list.

Step 6: Traverse and print the List Forward

Design an algorithm to start from the head of the list and print all nodes till the tail of the list.

Step 7: Traverse and print the List Backward

Design an algorithm to start from the tail of the list and print all nodes till the head of the list.

Step 8: Delete a Node

Design an algorithm to delete a node at a specific position in the doubly linked list. Write these steps in the notebook and provide the implementation.

Note: Be careful with edge cases, such as deleting at the head (position 1) or at the tail of the list.

Reversing the doubly linked list

Design an algorithm to reverse the doubly linked list. Write these steps in the notebook and provide the implementation.

Removing duplicate elements from the doubly linked list

Design an algorithm to remove duplicate elements from the doubly linked list. Write these steps in the notebook and provide the implementation.

Conclusion

In this lab session, we covered the basic steps to create and manage a doubly linked list. The ability to traverse in both directions offers significant advantages for certain use cases, and understanding how to manipulate pointers in such a structure is crucial for advanced programming.